CIS 4020 · DATA STRUCTURES

My Notes: Advanced Trees

Balanced search trees: 2-3 trees, 2-3-4 trees, and B-trees

A word before you read. These are my notes from years of teaching this unit, cleaned up so they are readable. For these notes, I have been using AI to help type these up, check for issues, and fill in explanations, which means a wrong or imprecise statement can slip in and survive longer than it should, and I do not fully trust the AI, so I am still editing. They are **not** meant to be a study guide, they are not gospel, and not everything here shows up in a lecture (and some lecture items are not here). If you find something that looks wrong, contradicts the book, or just does not behave the way I claim, **tell me**. You will be doing students and me a favor, and you will understand it better for having caught it. The textbook is the tiebreaker. We are a two-language course, thus I will do my best to show everything in Java **(CIS)** and C/C++ **(CS)**. The lecture drawings carry the full step-by-step walkthroughs; these notes pull out the rules and the moments that matter.

1. Why balance matters

Every operation on a binary search tree (search, insert, delete) follows one path from the root down toward a leaf. The cost of the operation is the length of that path, so the efficiency of a binary search tree depends entirely on its **height**.

The trouble is that the height depends on the order the items arrive in. Insert 40, 20, 60, 10, 30, 50, 70 and you get a tree three levels tall. Insert the same seven values in sorted order and every new value becomes the right child of the last one, giving a tree seven levels tall. That second tree is a linked list wearing a tree costume, and searching it is $O(n)$, not $O(\log n)$. The more one path outgrows the others, the more *unbalanced* the tree is and the closer its performance drifts toward linear.

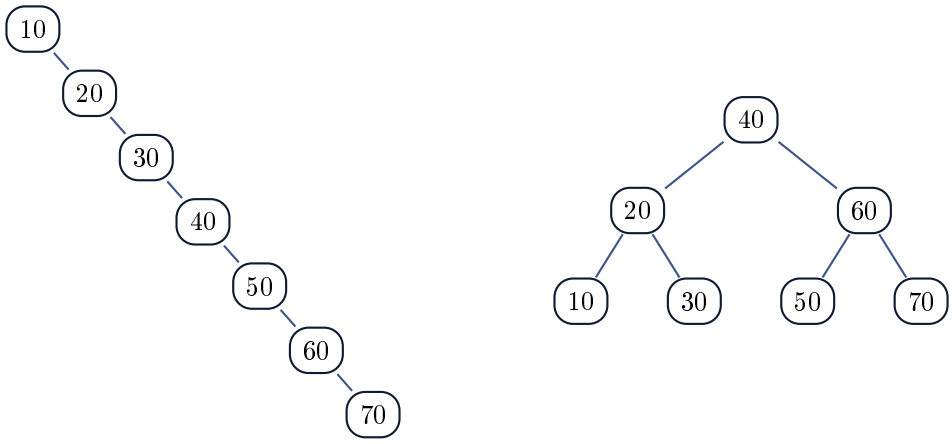


Figure 1: The same seven values in two binary search trees. On the left, inserted in sorted order (10, 20, 30, 40, 50, 60, 70), every node has only a right child and the tree is seven levels tall. On the right, inserted in the order 40, 20, 60, 10, 30, 50, 70, the tree is full and only three levels tall.

A **balanced search tree** fixes this by reorganizing itself during every insertion and deletion so that its height stays logarithmic no matter what order the items arrive in. There are several well-known families. This unit works through the first three and then looks at the others.

- 2-3 tree
- 2-3-4 tree
- B-tree
- Red-black tree and AVL tree

2. The 2-3 tree

The rules

A 2-3 tree is named for the two kinds of internal node it allows.

- A **2-node** holds one value and has exactly two children.
- A **3-node** holds two values and has exactly three children.
- A **leaf** holds one or two values and has no children.
- **All leaves are on the same level.** This is the rule that makes the tree balanced, and every algorithm below exists to protect it.

Notice the word *exactly*. A 2-node cannot have one child, and a 3-node cannot have two. Either a node is a leaf, or every child slot it has is filled.

The values inside a node are kept in order, and the children are ordered around them the same way a binary search tree orders its subtrees. For a 3-node holding a small value S and a large value L, everything in the left subtree is less than S, everything in the middle subtree is between S and L, and everything in the right subtree is greater than L.

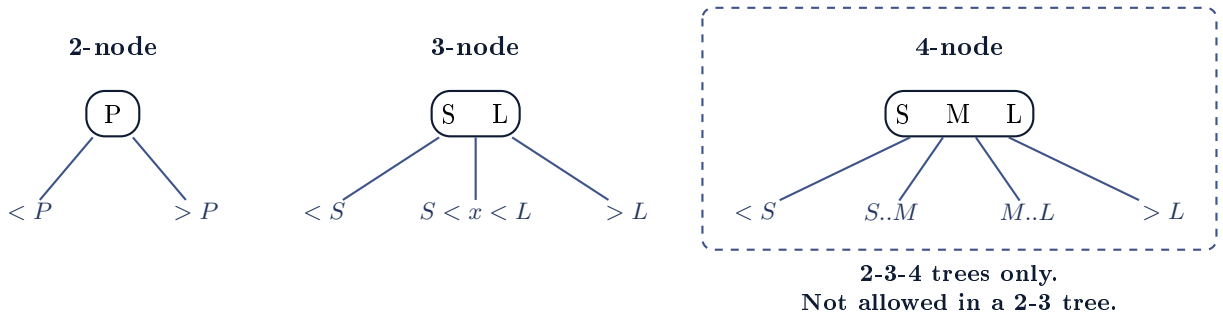


Figure 2: How values in each subtree compare with the values in the node. Left, a 2-node holding P: the left subtree holds values less than P and the right subtree holds values greater than P. Center, a 3-node holding S and L: left subtree less than S, middle subtree between S and L, right subtree greater than L. Right, inside a dashed box labeled "2-3-4 trees only": not allowed in a 2-3 tree, a 4-node holding S, M, and L (used by the 2-3-4 tree in Section 5), with values less than S, between S and M, between M and L, and greater than L.

A 2-3 tree never contains a 4-node. A node with three values exists only for a moment during insertion, and it is split right away.

So a 2-3 tree is **not** a binary search tree, because a node can have three children, but it **has the binary search tree property**: at every node, smaller values are to the left and larger values are to the right. That is enough to search it the same way.

Searching and traversing

To search, start at the root and compare the target with the one or two values in the node. If it matches, you are done. Otherwise the comparisons tell you which of the two or three children to move to, and you repeat. If you fall off the bottom of the tree, the value is not there.

An inorder traversal extends the binary search tree version with one extra visit:

- Binary search tree: left, visit, right.
- 2-3 tree, at a 3-node: left, visit small, middle, visit large, right.

Because of the ordering rule, an inorder traversal of a 2-3 tree visits every value in sorted order. That makes it the best check you have on a tree you built by hand: if the traversal is not sorted, something went wrong.

How tall can it be?

A 2-3 tree with n values is at its tallest when every node is a 2-node, which is just a full binary tree, and at its shortest when every node is a 3-node. That gives

$$\log_3(n+1) \;\le\; h \;\le\; \log_2(n+1).$$

A 2-3 tree is never taller than a minimum-height binary tree holding the same values. For a million values, the height is somewhere between 13 and 19 levels. An unbalanced binary search tree holding the same million values could be a million levels tall. Since search, insertion, and deletion each walk one path from root to leaf (and at most back up again), all three are $O(\log n)$.

3. Inserting into a 2-3 tree

The algorithm

1. Search for the new value. The search always ends at a **leaf**, and that is where the value goes. New values are never placed directly into an internal node.
2. If the leaf now holds two values, you are done.
3. If the leaf now holds three values (call them S, M, and L in order), it is too full and must **split**:
   - S and L become two separate nodes.
   - M, the middle value, moves **up** into the parent.
4. Moving M up may give the parent three values. If so, the parent splits the same way, and this time its four children are divided as well: the two leftmost go with S and the two rightmost go with L.
5. If the split reaches the **root**, the root splits into S and L, and M becomes a brand-new root above them. The tree is now one level taller.

Step 5 is the heart of the whole unit. A 2-3 tree grows *upward*, at the root, never downward at the leaves. When the root splits, every leaf moves one level deeper at the same time, so all leaves stay on the same level automatically.

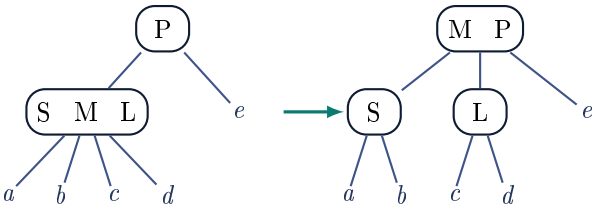


Figure 3: Splitting a node that has a parent. The overfull node S M L is the left child of P and has subtrees a, b, c, and d. After the split, M has moved up beside P, S keeps subtrees a and b, and L keeps subtrees c and d. If S M L is a leaf, the same picture applies with no subtrees a through d.

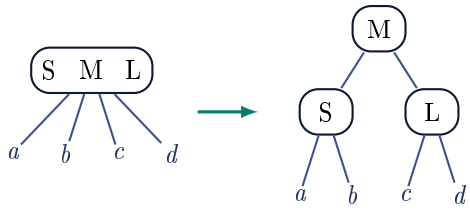


Figure 4: Splitting the root. The overfull root S M L with subtrees a through d becomes a new root M with two children, S (subtrees a and b) and L (subtrees c and d). This is the only way a 2-3 tree gains height.

The worked example: the moments that matter

In class we insert 42, 36, 16, 14, 32, 31, 40, 8, 21, 34, 20, 22, 50, 15 into an empty 2-3 tree, one value at a time, and the drawings show every step. Most insertions are quiet: the value lands in a leaf that has room and nothing else changes. The table lists only the insertions where something structural happens.

Table 1: Insertions in the 2-3 tree example that cause a split. Every insertion not listed simply adds the value to a leaf that already had room.

Insert	What happens
16	The single root leaf becomes 16 36 42 and splits. 36 becomes the new root. Height 2.
32	The leaf 14 16 32 splits and 16 moves up. The root becomes 16 36.
21	The leaf 21 31 32 splits and 31 moves up, making the root 16 31 36. The root splits too, and 31 becomes the new root. Height 3. One insertion, two splits.
22	The leaf 20 21 22 splits and 21 moves up beside 16.
50	The leaf 40 42 50 splits and 42 moves up beside 36.
15	The leaf 8 14 15 splits and 14 moves up, making 14 16 21, which splits and sends 16 up into the root. The root becomes 16 31, which has room, so the height stays 3.

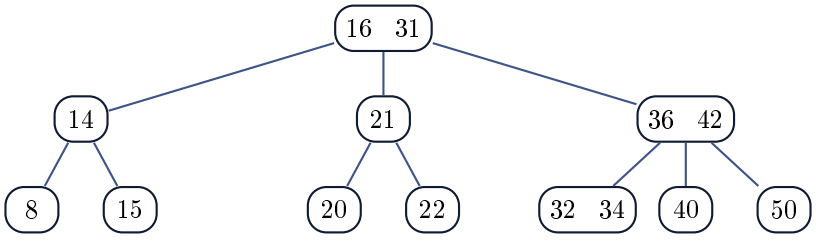


Figure 5: The final 2-3 tree from the worked example. The root holds 16 and 31. Its left child 14 has leaves 8 and 15. Its middle child 21 has leaves 20 and 22. Its right child holds 36 and 42 and has leaves 32 34, 40, and 50. All seven leaves are on level 3.

Checking it with an inorder traversal gives 8, 14, 15, 16, 20, 21, 22, 31, 32, 34, 36, 40, 42, 50, which is sorted, so the tree is correct. Fourteen values fit in three levels; the same values inserted into a plain binary search tree in this order would need seven.

4. Deleting from a 2-3 tree

Deletion is insertion run backward. Insertion can leave a node with too many values, which we fix by splitting. Deletion can leave a node with *no* values, which we fix by borrowing from a neighbor (**redistribution**) or by combining with one (**merging**). Redistribution fixes the problem locally, while merging fixes it locally but may push the problem up to the parent.

The algorithm

1. Locate the node that holds the value I you want to delete.
2. If that node is not a leaf, find the **inorder successor** of I (the smallest value larger than I) and swap the two. The inorder successor of a value in an internal node is always in a leaf, so now the deletion always happens at a leaf. This is the same trick you saw for binary search trees.
3. Remove I from the leaf. If the leaf still holds another value, you are done.
4. Otherwise the leaf is now **empty**, and you repair it using the cases in Table 2. A merge can leave the *parent* empty, in which case you repair the parent the same way, and so on up the tree.

Table 2: Repairing an empty node in a 2-3 tree. The letters match the deletion figure in the textbook and in the lecture drawings.

Case	Situation	Repair
(a)	Empty leaf, a sibling has two values	**Redistribute.** The parent value moves down into the empty leaf, and the sibling's nearer value moves up to replace it.
(b)	Empty leaf, the sibling has one value	**Merge.** The parent value moves down into the sibling, and the empty leaf is removed. The parent may now be empty.
(c)	Empty internal node, a sibling has two values	**Redistribute** as in (a), and the sibling also hands over its nearest child, so the empty node ends up with two children again.
(d)	Empty internal node, the sibling has one value	**Merge** as in (b), and the empty node's only child moves over to the merged node. The parent may now be empty.
(e)	The root is empty	**Delete the root.** Its only child becomes the new root. The tree is now one level shorter.

Case (e) is the mirror image of a root split: the only way a 2-3 tree loses height is at the root, so once again every leaf moves together and they all stay on the same level.

The worked example: deleting 13

In class we delete 13 from a small tree whose root is 10, with children 8 (leaves 7 and 9) and 12 (leaves 11 and 13). Removing 13 empties its leaf, and the repairs then climb all the way to the root.

1. The empty leaf's sibling, 11, has only one value, so case (b) applies: 12 moves down and merges with 11, giving the leaf 11 12. The node that held 12 is now empty.
2. That empty internal node's sibling, 8, also has only one value, so case (d) applies: 10 moves

down and merges with 8, giving 8 10, and the leaf 11 12 moves over to become its right child. The root is now empty.

3. Case (e) applies: the empty root is deleted and 8 10 becomes the new root.



Figure 6: Deleting 13. Before, the tree has three levels: root 10, internal nodes 8 and 12, and leaves 7, 9, 11, and 13. After two merges and the removal of the empty root, the tree has two levels: root 8 10 with leaves 7, 9, and 11 12.

One deletion, two merges, and the tree lost a level. That chain reaction is the deletion version of the cascading split you saw when we inserted 21.

5. The 2-3-4 tree

The rules

A 2-3-4 tree adds one more node type to the 2-3 tree.

- A **2-node** holds one value and has exactly two children.
- A **3-node** holds two values and has exactly three children.
- A **4-node** holds three values and has exactly four children.
- A **leaf** holds one, two, or three values.
- All leaves are on the same level.

The ordering rule extends in the obvious way (see the 4-node in Figure 2), and searching and inorder traversal work just like they do in a 2-3 tree, with one more comparison and one more visit at a 4-node. Like the 2-3 tree, it is not a binary search tree, but it has the binary search tree property.

A 2-3-4 tree is still split at its middle value, but now a node is "full" when it already holds three values. The interesting new question is *when* to split it.

Preemptive split (top-down)

With a **preemptive split**, you split every 4-node you meet on the way *down* the insertion path, before you ever reach the leaf. The middle value moves up, and the outer two values become 2-nodes. Because the node above was already processed on the way down, it is guaranteed not to be a 4-node, so it always has room for the value moving up. That means a split never cascades, and the whole insertion is a single pass from root to leaf.

There are only three situations to handle, all shown in the lecture drawings:

- A **4-node root** splits into a new 2-node root with two 2-node children. The tree grows one level.
- A **4-node whose parent is a 2-node** splits, and the parent becomes a 3-node.

- A **4-node whose parent is a 3-node** splits, and the parent becomes a 4-node.

The parent can never be a 4-node, because a 4-node parent would already have been split.

The worked example: preemptive

In class we insert 30, 10, 60, 20, 50, 40, 70, 80, 15, 90, 19. Highlights:

- After 30, 10, and 60 the root is the 4-node 10 30 60. Inserting 20 splits it *before* 20 goes in: 30 becomes the new root, and 20 joins 10.
- Inserting 70 meets the full leaf 40 50 60 on the way down, so it splits first and 50 joins the root. Inserting 90 does the same to 60 70 80, sending 70 up.
- Inserting 19 is the big one. The root 30 50 70 is a 4-node, so it splits first and the tree grows to three levels. Then the leaf 10 15 20 is a 4-node, so it splits and 15 moves up. Only then does 19 go into its leaf, beside 20.

Step back for a moment. What if we insert 100 instead of 19? Its leaf, 80 90, has plenty of room. It does not matter: the root 30 50 70 is a 4-node sitting on the insertion path, so a preemptive split splits it anyway. The tree grows to three levels even though, strictly speaking, it did not need to. That is the price of the single pass.

Non-preemptive split (bottom-up)

The alternative is to work exactly like the 2-3 tree: put the value in the leaf first, let the leaf temporarily hold **four** values, and split only then, sending a value up and letting the split cascade upward if the parent overflows too.

With four values there is no single middle. In this course, the value that moves up is always the **second** value. For example, the overfull leaf 10 20 30 60 sends 20 up and leaves 10 and 30 60 behind. Other sources sometimes send the third value up instead, which builds a different tree, so follow the lecture convention on assignments and exams.

Running the same insertion sequence with non-preemptive splits produces a different, and in this case shorter, tree. Inserting 100 instead of 19 at the end causes a leaf split that cascades to the root, and only then does the tree grow to three levels.



Figure 7: The same eleven values inserted into a 2-3-4 tree two ways. Left, preemptive splitting: root 50; its left child 15 30 has leaves 10, 19 20, and 40; its right child 70 has leaves 60 and 80 90. Three levels. Right, non-preemptive splitting: root 20 40 60 with leaves 10 15 19, 30, 50, and 70 80 90. Two levels.

Table 3: Comparing the two splitting strategies for a 2-3-4 tree.

	Preemptive (top-down)	Non-preemptive (bottom-up)
When it splits	Every 4-node on the way down, needed or not	Only a node that actually overflows
Largest node, even briefly	Three values	Four values, until the split
Can a split cascade?	No, the parent always has room	Yes, all the way to the root
Passes through the tree	One, root to leaf	Down to the leaf, then possibly back up

Both strategies produce a valid 2-3-4 tree and keep every operation $O(\log n)$, but they build different trees from the same values, and neither one always produces the shorter tree. The textbook uses the preemptive version. On assignments and exams, use the strategy the instructions name.

Deleting from a 2-3-4 tree

Deletion from a 2-3 tree removes the value first and then repairs empty nodes on the way back *up*, sometimes all the way to the root. Deletion from a 2-3-4 tree, as the book describes it, works the other way around. It makes repairs on the way *down*, so that when it reaches the leaf, the leaf always has a value to spare and nothing needs repairing afterward. It is the deletion version of preemptive splitting: one pass from root to leaf, with no backing up.

What stays the same as a 2-3 tree.

1. Locate the value you want to delete.
2. If it is in an internal node, swap it with its **inorder successor**, so that the deletion always happens at a leaf.
3. If that leaf is a 3-node or a 4-node, simply remove the value. The leaf still has at least one value left.

The only trouble is a leaf that is a 2-node, because removing its only value would leave it empty. The rest of the algorithm exists to make sure that never happens. The textbook describes these cases and leaves the full details of the deletion process as an exercise; the rest of this section works through them.

Never step into a 2-node

As you search for the value, and then for its inorder successor, look at each node *before* you move into it. If it is a 2-node, and it is not the root, turn it into a 3-node or a 4-node first. To do that, look at its **nearest sibling**: the left sibling, unless the node is a left child, in which case it is the right sibling.

The nearest sibling is a 3-node or a 4-node: borrow. The parent value between the two nodes moves down into the 2-node, which becomes a 3-node. The sibling's nearest value moves up to take its place, and the sibling's nearest child moves over with it. This is the same redistribution you saw in the 2-3 tree, done on the way down instead of on the way back up.

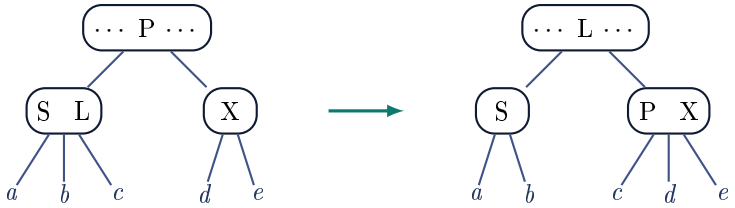


Figure 8: Borrowing from the nearest sibling. Before: the parent value P sits between the sibling S L, with subtrees a, b, and c, and the 2-node X, with subtrees d and e. After: P has moved down beside X, L has moved up into the parent, and subtree c has moved over to P X. If the sibling is a 4-node, the same thing happens and it is left as a 3-node. When the sibling is on the right, the picture is mirrored.

The nearest sibling is a 2-node: merge. The 2-node, the parent value between them, and the sibling combine into one 4-node. That is one value plus one value plus one value, so the merged node is always a 4-node. The parent loses one value. This is exactly a 4-node split run in reverse, which is why the book describes these cases as its split figures applied "in reverse."

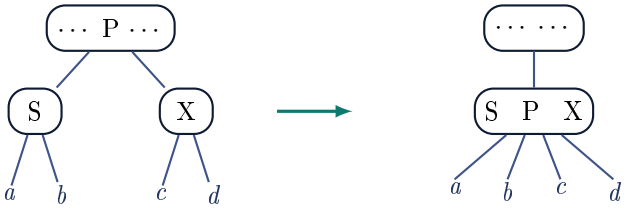


Figure 9: Merging with the nearest sibling. Before: the parent value P sits between two 2-nodes, S with subtrees a and b, and X with subtrees c and d. After: S, P, and X form one 4-node with subtrees a, b, c, and d, and the parent has one fewer value.

The parent always has a value to give. It was already turned into a 3-node or a 4-node one step earlier, on the way down, so the only parent that can still be a 2-node is the root.

Table 4: What a merge does to the parent.

The parent was	After the merge
a 4-node	It becomes a 3-node.
a 3-node	It becomes a 2-node.
a 2-node	This can only be the root. See the next subsection.

The root. If the root is a 2-node and *both* of its children are 2-nodes, all three merge into a single 4-node root. This is the only way a 2-3-4 tree gets shorter, and it is the mirror image of splitting the root.

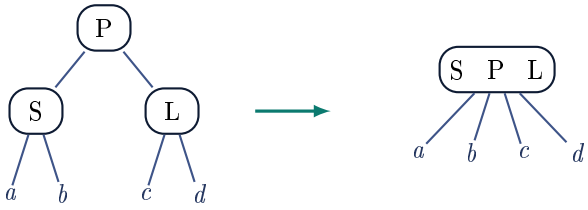


Figure 10: Merging at the root. Before: a 2-node root P with two 2-node children, S with subtrees a and b, and L with subtrees c and d. After: one 4-node root S P L with subtrees a, b, c, and d. The tree is one level shorter.

Where the 4-node makes deletion easier

- **A 4-node never needs fixing.** It has three values, so it is never close to running out.
- **A 4-node is the best sibling to borrow from.** It has two values to spare.
- **A merge always produces a 4-node, never anything bigger.** You never merge *with* a 4-node, or even a 3-node, because if the nearest sibling has a spare value you borrow instead.
- **The bookkeeping is the hard part.** In a 4-node parent, which sibling is "nearest," and which parent value comes down, depends on which of the four children you are moving into. Numbering the children 1 through 4 on a drawing helps.

Worked example: deleting from a 2-3-4 tree

We start from the 2-3-4 tree we built with preemptive splitting (the left tree in Figure 7) and delete 40, 60, 50, and 70.

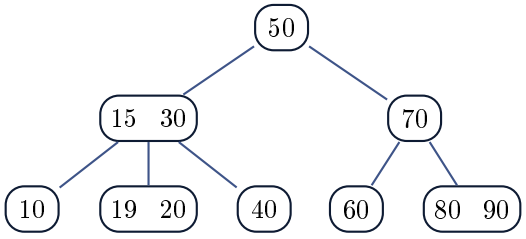


Figure 11: The starting 2-3-4 tree. Root 50. Level 2: 15 30; 70. Level 3: 10; 19 20; 40; 60; 80 90.

Delete 40: borrow from a 3-node. The root 50 has one 3-node child, so it is not merged. The next node, 15 30, is not a 2-node. The next node is the leaf 40, a 2-node. It is the right child, so its nearest sibling is the left sibling, 19 20, a 3-node: borrow. 30 moves down beside 40, and 20 moves up into the parent. Now the leaf 30 40 has a spare value, and 40 is removed.

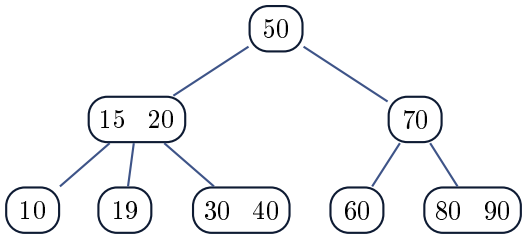


Figure 12: After borrowing, before removing 40. Root 50. Level 2: 15 20; 70. Level 3: 10; 19; 30 40; 60; 80 90.

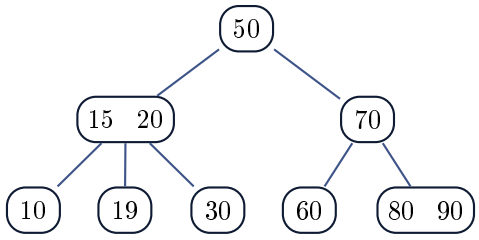


Figure 13: After deleting 40. Root 50. Level 2: 15 20; 70. Level 3: 10; 19; 30; 60; 80 90.

Delete 60: a borrow, then a merge. The root 50 is a 2-node, but its left child 15 20 is a 3-node, so the root is not merged. The next node, 70, is a 2-node and a right child, so its nearest sibling is the left sibling, 15 20: borrow. 50 moves down beside 70, 20 moves up into the root, and the leaf 30 moves over to 50 70.

The next node is the leaf 60, a 2-node. It is a middle child, so its nearest sibling is the left sibling, 30, which is also a 2-node: merge. 30, 50, and 60 become one 4-node, and the parent 50 70 becomes 70. Now 60 is removed from a leaf that has values to spare.

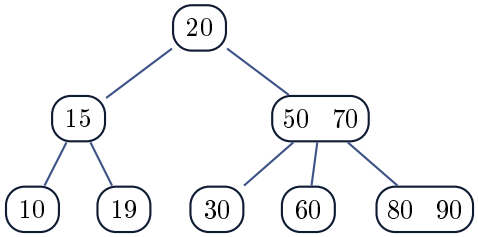


Figure 14: After the borrow at 70. Root 20. Level 2: 15; 50 70. Level 3: 10; 19; 30; 60; 80 90.

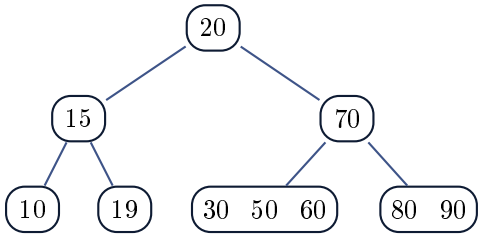


Figure 15: After the merge at 60. Root 20. Level 2: 15; 70. Level 3: 10; 19; 30 50 60; 80 90.

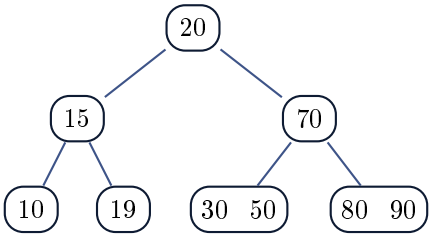


Figure 16: After deleting 60. Root 20. Level 2: 15; 70. Level 3: 10; 19; 30 50; 80 90.

Delete 50: merge at the root. The root 20 is a 2-node, and both of its children, 15 and 70, are 2-nodes, so all three merge into the root 15 20 70. The tree drops to two levels. Then 50 is removed from the leaf 30 50.

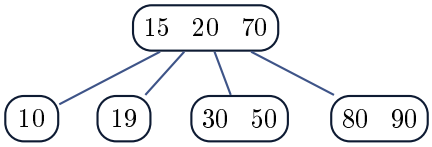


Figure 17: After the root merge. Root 15 20 70. Level 2: 10; 19; 30 50; 80 90.

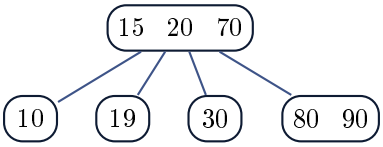


Figure 18: After deleting 50. Root 15 20 70. Level 2: 10; 19; 30; 80 90.

Delete 70: swap with the inorder successor. 70 is in the root, which is an internal node. The search for its inorder successor goes to the child just right of 70, the leaf 80 90, which is a 3-node and needs no change. The successor, 80, replaces 70, and 80 is removed from the leaf, leaving 90.

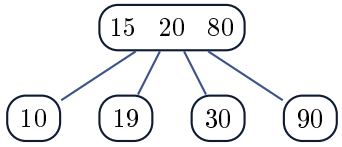


Figure 19: After deleting 70. Root 15 20 80. Level 2: 10; 19; 30; 90.

Watch out: the value can move. A borrow or a merge can pull the value you are deleting down a level, when that value is the parent value that moves. When that happens, keep searching from wherever the value went.

For example, delete 20 from the tree we reached after deleting 50 (root 15 20 70, leaves 10, 19, 30, and 80 90). 20 is in the root, so the search for its successor moves toward the leaf 30, a 2-node. Its nearest sibling, the left sibling 19, is also a 2-node: merge. The value that comes down from the parent is 20 itself, so the merged leaf is 19 20 30, and 20 is now in a leaf. No swap is needed; 20 is simply removed from that leaf.

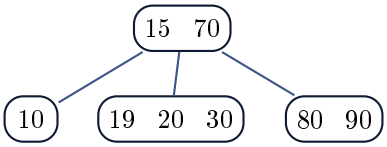


Figure 20: After the merge: 20 has moved down. Root 15 70. Level 2: 10; 19 20 30; 80 90.

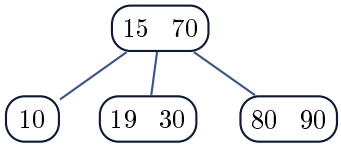


Figure 21: After deleting 20. Root 15 70. Level 2: 10; 19 30; 80 90.

Some sources delete from a 2-3-4 tree bottom-up, the way a 2-3 tree does, which builds different trees, so follow the book's top-down method on assignments and exams.

6. The B-tree

The generalization

Look at what changed between the 2-3 tree and the 2-3-4 tree: the largest node grew by one value and one child, and every rule came along unchanged. Nothing stops us from going further. A **B-tree** lets the largest node have any number of children, m, which is called the **order** (some sources say **degree**) of the tree.

- m is the maximum number of children a node can have.
- A node can hold at most $m - 1$ keys (values).
- $m = 3$ is a 2-3 tree, and $m = 4$ is a 2-3-4 tree.
- $m = 2$ would give every node one key and two children, the shape of a binary tree, but with every leaf forced onto the same level.

Donald Knuth gave the properties of a B-tree of order m their standard form:

1. Every node has at most m children.
2. Every non-leaf node, except the root, has at least $\lceil m/2 \rceil$ children. In other words, a node can never be less than about half full.

3. The root has at least two children if it is not a leaf.
4. A non-leaf node with k children holds $k - 1$ keys.
5. All leaves appear on the same level. (Knuth counts the empty spots below the bottom row of nodes as the leaves, which is why his version says they "carry no information." In our drawings, it simply means the bottom row is all on one level.)

A warning about vocabulary: books do not agree on what "order" means. Some define it by the maximum number of children (our definition), some by the maximum number of keys, and some by the *minimum* number of children. When you read about B-trees anywhere else, check which definition the author is using before you trust any numbers.

Why B-trees exist

Rudolf Bayer and Edward McCreight introduced the B-tree around 1970 to manage the index pages of large random access files. The problem they were solving is that reading from a disk is enormously slower than reading from memory, and a disk is read in fixed-size blocks, or pages, not one value at a time. If each node is sized to fill one page, then every node you visit costs one disk read, and the number of reads is the height of the tree.

Large nodes make for very short trees. With $m = 100$, each node holds up to 99 keys, and a tree only three levels tall can hold close to a million keys when its nodes are full. Even with every node only half full, a million keys never needs more than four levels. Finding any one of a million records in four disk reads, instead of the twenty a balanced binary tree would need, is why variants of the B-tree are still the standard index in relational databases and appear in many file systems. Some write-heavy databases use a different structure, the LSM tree (log-structured merge tree), which trades slower reads for much faster writes.

The worked example: order 6

In class we insert the same eleven values as the 2-3-4 example (30, 10, 60, 20, 50, 40, 70, 80, 15, 90, 19) into a B-tree with $m = 6$, so each node holds at most five keys. In the demonstration, a full node is split *before* the new key goes in: its middle key moves up and the new key then goes into whichever half it belongs in.

- The first five values all fit in the root: 10 20 30 50 60.
- Inserting 40 finds the root full, so it splits. 30 becomes the new root, and 40 joins 50 60.
- 70, 80, and 15 fit without trouble, which fills the right leaf: 40 50 60 70 80.
- Inserting 90 finds that leaf full, so it splits and 60 moves up beside 30. 90 then joins 70 80.
- 19 slides into the left leaf.

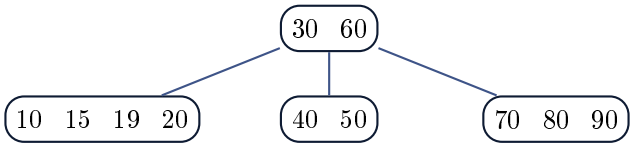


Figure 22: The final B-tree of order 6. The root holds 30 and 60, with three leaves: 10 15 19 20, 40 50, and 70 80 90. Every node holds between two and five keys.

The same eleven values took three levels in the preemptive 2-3-4 tree and only two here. That is the B-tree trade in miniature: bigger nodes, fewer levels.

The same example, overflow then split

The demonstration splits a full node before the new key goes in. A B-tree can also be built the non-preemptive way, like the 2-3-4 tree in Section 5: the new key goes in first, the node holds one key too many for a moment, and then it splits. For a B-tree of order m:

1. Search for the new key. The search ends at a leaf, and the key goes there.
2. If the leaf now holds $m - 1$ keys or fewer, you are done.
3. If the leaf now holds m keys, it has **overflowed**. Split it: the middle key moves up into the parent, the keys to its left form one node, and the keys to its right form another.
4. If the parent now overflows, split it the same way, dividing its children between the two halves. If the split reaches the root, the middle key becomes a new root and the tree grows one level.

With $m = 6$, a node holds at most five keys, so an overflowing node holds six. Six keys have no single middle. As with the 2-3-4 tree, the key that moves up is always the **lower** middle key, the third of the six. The left half keeps two keys and the right half keeps three. Both halves meet the B-tree minimum of $\lceil 6/2 \rceil - 1 = 2$ keys.

Table 5: Building the order-6 B-tree with overflow then split. Insertions not listed simply add the key to a leaf that had room.

Insert	What happens
30, 10, 60, 20, 50	All five fit in the root, which is also a leaf: 10 20 30 50 60.
40	40 goes in first, giving six keys: 10 20 30 40 50 60. The node overflows and splits. The third key, 30, becomes the new root, with leaves 10 20 and 40 50 60.
70, 80, 15	These fit. The right leaf fills up to 40 50 60 70 80.
90	90 goes in first, giving 40 50 60 70 80 90. The leaf overflows and splits. The third key, 60, moves up beside 30, leaving leaves 40 50 and 70 80 90.
19	Fits in the left leaf: 10 15 19 20.

This is the same tree as Figure 22. That is not a coincidence for this sequence. In both splits, the new key landed above the middle of the full node, so the key that moved up was the same either way.

Where the two approaches part ways. The two approaches give different trees when the new key lands *below* the middle of a full node. Go back to the moment the root first fills up with 10 20 30 50 60, and suppose the next key is 5 instead of 40.

- **Preemptive.** The full node splits before 5 arrives. Its middle key among the five, 30, moves up, leaving 10 20 and 50 60. Then 5 goes into the left leaf, giving 5 10 20.
- **Overflow then split.** 5 goes in first, giving 5 10 20 30 50 60. The third of the six keys, now 20, moves up, leaving 5 10 and 30 50 60.



Figure 23: Inserting 5 into the full root 10 20 30 50 60 of an order-6 B-tree. Left, preemptive: root 30 with leaves 5 10 20 and 50 60. Right, overflow then split: root 20 with leaves 5 10 and 30 50 60. Both are valid B-trees of order 6. They differ because one approach chose the key to promote before seeing 5 and the other chose after.

The trade-offs are the ones in Table 3, with $m - 1$ keys in place of three and m in place of four. For data on disk, the preemptive pass reads each node once and never revisits it, while a cascade in the overflow approach walks back up and rewrites nodes already read. Both keep every operation $O(\log n)$ and always produce a valid B-tree. They build different trees, so use the approach the instructions name, for every insertion in the sequence.

Comparing the family

Table 6: The three trees in this unit as members of one family.

Tree	Order m	Keys per node	Children per internal node
2-3 tree	3	1 or 2	2 or 3
2-3-4 tree	4	1 to 3	2 to 4
B-tree	m	$\lceil m/2 \rceil - 1$ to $m - 1$ (root may hold 1)	$\lceil m/2 \rceil$ to m (root may have 2)

In all three, every leaf is on the same level, the tree grows and shrinks only at the root, and search, insertion, and deletion are all $O(\log n)$.

7. Where this goes next

Each of these builds on the ideas in this unit.

Unlike the 2-3 and 2-3-4 trees, the red-black tree and the AVL tree are true binary search trees: every node holds one value and has at most two children. They stay balanced by storing a little extra information in each node, a color or a height, and restructuring with rotations.

- **Red-black tree.** A binary search tree that represents a 2-3-4 tree. Each 3-node or 4-node is broken into small groups of ordinary binary nodes, and the links inside a group are colored red so the original node can be recognized. You get the 2-3-4 tree's balance while keeping simple two-child nodes. Java's TreeMap and most C++ std::map implementations are red-black trees.

  Figure 24 stores the preemptive 2-3-4 tree from Figure 7 as a red-black tree. Each 2-node stays a single node. Each 3-node becomes two binary nodes joined by a red link. A 4-node, had there been one, would become its middle value with red links to both outer values. Count only the black links on any path from the root down to the bottom and you get the same number every time, two here. Those two black links are the two edges on every path from the root to a leaf in the 2-3-4 tree, which is the red-black version of "all leaves on the same level." A 3-node can be split either way, with either value on top, so other books may draw a slightly different red-black tree for the same data.

  Many books, and most code, color the **nodes** instead of the links. The two views say the same thing: a node is red exactly when the link coming down into it is red, and the root, which has

no link above it, is always black. In node terms the rules read: the root is black; a red node never has a red child, because two red links in a row would mean a 2-3-4 node with too many values; and every path from the root down passes through the same number of black nodes. In Figure 24 that number is three, for example 50, 30, and 40. That is one more than the count of black links, because the black root is counted as a node but has no link above it.

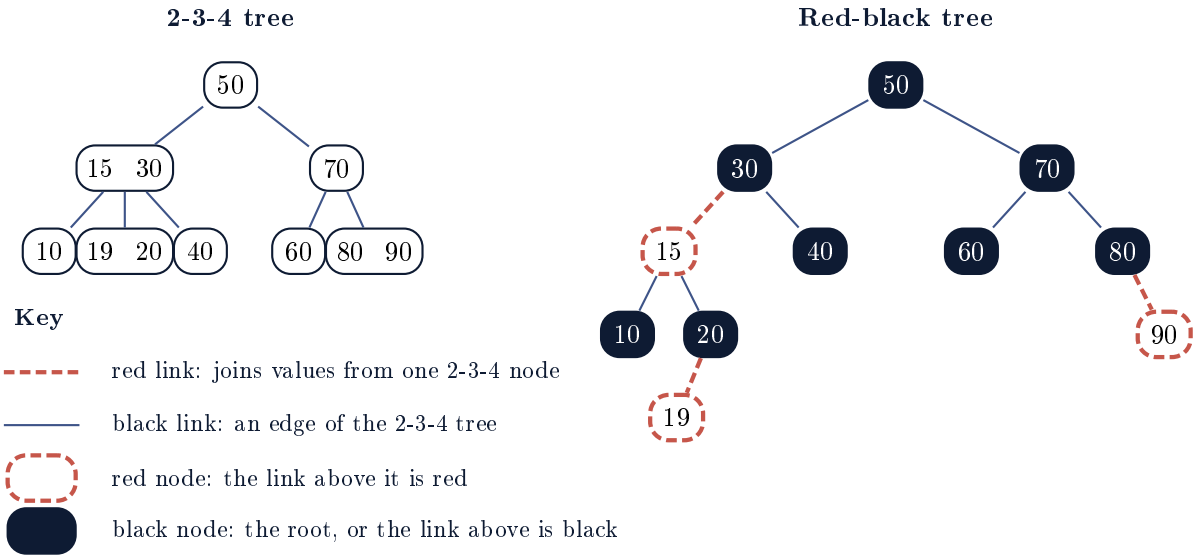


Figure 24: The same eleven values as a 2-3-4 tree and as a red-black tree. Left, the 2-3-4 tree: root 50; its children are 15 30 and 70; the leaves are 10, 19 20, 40, 60, and 80 90. Right, the red-black tree: root 50 with children 30 and 70. Node 30 has a red link to 15 and a black link to 40. Node 15 has children 10 and 20, and 20 has a red link to its left child 19. Node 70 has children 60 and 80, and 80 has a red link to its right child 90. Red links are drawn thick and dashed. Every other link is black. The nodes are colored to match: the red nodes 15, 19, and 90 have a thick dashed outline and a white fill, and every other node is black, drawn with a dark fill and white numbers.

- **AVL tree.** A binary search tree in which the heights of the two subtrees of every node differ by at most one. When an insertion or deletion breaks that rule, the tree repairs itself with *rotations*. Named for its inventors, Adelson-Velsky and Landis.

  Figure 25 inserts 10, 20, 30, 40, 50, 60 in sorted order, the same kind of input that turned the plain binary search tree in Figure 1 into a chain. After 30 goes in, the node 10 has an empty left subtree and a right subtree two levels tall, so a left rotation makes 20 the parent of 10 and 30. Two more rotations happen along the way, one after 50 and one after 60. The final tree is not perfectly full, since 50 has only one child, but at every node the two subtree heights differ by at most one, so it is a valid AVL tree.

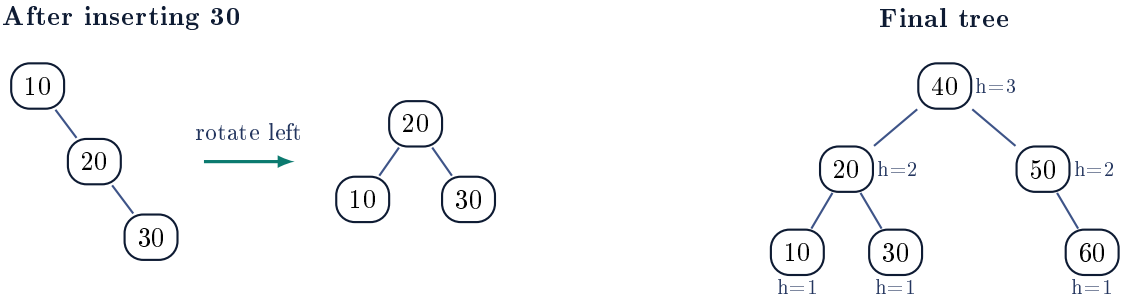


Figure 25: An AVL tree built from 10, 20, 30, 40, 50, 60 inserted in order. Left, after 30 is inserted the tree is the chain 10, 20, 30, which is out of balance at 10; a left rotation gives root 20 with children 10 and 30. Right, the final tree after all six insertions and three rotations: root 40, height 3; its left child 20, height 2, has leaves 10 and 30; its right child 50, height 2, has only a right child, 60. Each leaf has height 1.

- **B+ tree.** A B-tree variant that keeps all the actual data in the leaves and uses the internal nodes only to guide the search. The leaves are linked together, so reading a range of keys in order is fast. This is the version most relational databases use for their indexes.
- **R-tree.** A tree for spatial data, such as points, rectangles, and polygons on a map. Nearby objects are grouped under bounding rectangles, so a question like "what is near this location" only searches the parts of the tree that could possibly answer it.

8. Common mistakes

- **Placing a new value in an internal node.** Insertion always starts at a leaf. Values only reach internal nodes by being pushed up during a split.
- **Growing the tree downward.** If you find yourself hanging a new leaf below an existing leaf, stop. A balanced tree only gains height when the root splits.
- **Promoting the wrong value.** In a 2-3 tree, it is the middle of the three values that moves up, not the new value. The new value only moves up if it happens to be the middle one.
- **Losing children during a split.** When an internal node splits, its four children must be divided two and two between S and L. Count the children before and after; the number should not change.
- **Calling a 2-3 tree a binary search tree.** It keeps the ordering property, but a binary search tree has at most two children per node.
- **Using the wrong split strategy in a 2-3-4 tree.** Preemptive and non-preemptive splitting build different trees. Check which one the instructions ask for, and use it for every insertion in the sequence.
- **Skipping the case check.** Before you repair an empty node, look at its sibling and decide which case in Table 2 applies. Merging with a sibling that has two values leaves an overfull node (see Figure 26).
- **Mixing up keys and children in a B-tree.** A node of order m has up to m children but only $m - 1$ keys.

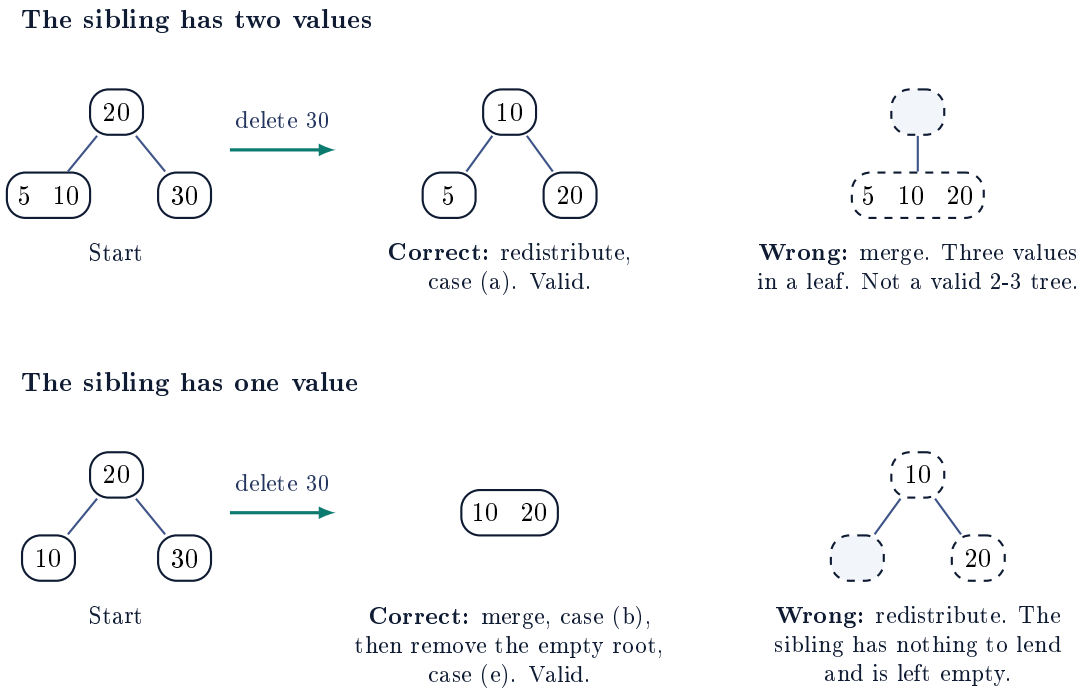


Figure 26: The sibling decides the case. Top row, the sibling has two values: starting from root 20 with leaves 5 10 and 30, deleting 30 and redistributing (case a) gives root 10 with leaves 5 and 20, which is valid. Merging instead pushes 20 down into the sibling, giving the leaf 5 10 20 under an empty root, which is not a valid 2-3 tree; it is drawn with dashed outlines. Bottom row, the sibling has one value: starting from root 20 with leaves 10 and 30, deleting 30 and merging (case b), then removing the empty root (case e), gives the single node 10 20, which is valid. Redistributing instead moves 10 up and 20 down, leaving root 10 with an empty left leaf and the leaf 20, which is not valid; it is drawn with dashed outlines.

9. Where this lives in the textbooks

The two books split this material differently, so it is worth knowing where to look.

- **Java, 3rd edition (Prichard and Carrano).** Chapter 13, Advanced Implementations of Tables, Section 13.1, Balanced Search Trees, covers 2-3 trees, 2-3-4 trees, red-black trees, and AVL trees. B-trees appear in Chapter 15, External Methods, under External Tables.
- **C/C++, 8th edition (Carrano and Henry).** Chapter 25, 2-3 Trees, and Chapter 26, Advanced Search Trees, which covers 2-3-4 trees and red-black trees. AVL trees appear earlier, in Chapter 19, Binary Search Trees. B-trees appear in Chapter 29, Processing Data in External Storage.

10. Further reading

These sources back up the discussion of where B-trees and LSM trees are used today.

- Alex Petrov, "Algorithms Behind Modern Storage Systems," *ACM Queue* 16(2), 2018. `https://queue.acm.org/detail.cfm?id=3220266`
- Sid Choudhury, "A Busy Developer's Guide to Database Storage Engines: The Basics," Yugabyte blog, 2018. `https://www.yugabyte.com/blog/a-busy-developers-guide-to-database-storage-engines-the-basics`

- ByteByteGo, "B-Tree vs. LSM-Tree." https://bytebytego.com/guides/b-tree-vs/
- Sylvain Tiset, "The Great Storage Engine Debate: B-Trees vs LSM-Trees," Medium, April 2026. https://medium.com/@sylvain.tiset/the-great-storage-engine-debate-b-trees-vs-lsm-trees-35d6975f2ede